Maker-Checker Approach for Quarkus Microservices with Apicurio Schema Registry

Schema-Centric Architecture, GitLab CI/CD, Kubernetes Deployment, and Fleet-Scale Migration

July 2026

Contents

1. Overview

This document describes a maker-checker (four-eyes) governance approach for schema-centric artifacts (`.avsc` Avro, Protobuf, OpenAPI) used across Quarkus microservices, integrated with Apicurio Schema Registry, deployed to Kubernetes, with source hosted in GitLab and delivered via GitLab CI/CD pipelines. It also covers the transaction outbox pattern for reliable event publishing, and a practical strategy for retrofitting this across a fleet of 700 existing microservices **without modifying their source code**.

2. Repository Structure

Schemas are kept in a dedicated repository, separate from service code:

```
schema-registry/
├── avro/
│   └── order-events/OrderCreated.avsc
├── protobuf/
│   └── payment/Payment.proto
├── openapi/
│   └── order-service/openapi.yaml
├── compatibility-rules.yaml
└── .gitlab-ci.yml
```

Service repos (Quarkus microservices) reference schema versions via a common library dependency, not raw schema files.

3. Maker-Checker via GitLab Merge Request Workflow

- **Maker**: Developer edits a `.avsc` / `.proto` / `.yaml` file and opens a merge request (MR).
- **CI validation stage** (automatic, on MR):
  - Schema syntax lint (avro-tools, protoc, spectral for OpenAPI)
  - Compatibility check against Apicurio Registry (dry-run using Apicurio's `/compatibility` endpoint against the target group/artifact before merge — reject on rule violation)
  - Diff annotation posted as an MR comment describing field-level changes
- **Checker (approval gate)**: Enforced via GitLab protected branches and required approvers (e.g. a `schema-owners` CODEOWNERS group), minimum 1–2 approvals, with "prevent approval by author" enabled so the approver can never be the same person as the maker.
- Merging triggers registration into Apicurio.

4. Apicurio Registry Integration

- Run separate Apicurio groups per environment (`/groups/dev`, `/groups/staging`, `/groups/prod`) either as distinct instances or logical groups within one instance.
- Set registry-level compatibility and validity rules so Apicurio enforces them server-side — defense in depth even if CI is bypassed.
- CI pipeline (post-merge, on `main`):
  1. Register/update the artifact in the dev group via the Apicurio REST API
  2. Promote to staging/production only via a manual, gated CI job — the second maker-checker gate, at the registry-promotion level, distinct from code-review approval
  3. Tag the Apicurio artifact version with the Git commit SHA for traceability

4.1 Compatibility Rule Cheat-Sheet

Mode	Meaning	Use when
`BACKWARD`	New schema can read data written with the old schema	Consumers upgrade before producers

Mode	Meaning	Use when
`BACKWARD_TRANSITIVE`	Backward-compatible with all previous versions, not just the last	Long-lived topics with many historic versions in flight
`FORWARD`	Old schema can read data written with the new schema	Producers upgrade before consumers
`FULL`	Both backward and forward	Independent team release cadences
`NONE`	No enforcement	Sandbox/dev experimentation only

For a common library shared across many independently-released services, `FULL` or `BACKWARD_TRANSITIVE` is the safer default on the shared events group.

5. Property-Only Changes

Non-schema property changes are routed through a lighter MR path (still requiring approval, but skipping the Apicurio compatibility stage). GitLab CI `rules:changes` conditionally trigger only the relevant pipeline stages depending on which paths changed (`avro/**`, `protobuf/**`, `openapi/**` versus `src/main/resources/application*.yaml`).

6. Common Library (Avro / Kafka Serializer)

A shared `event-schemas-lib`:

- Generates POJOs from `.avsc` files (Avro codegen)
- Bundles the Apicurio Kafka serializer/deserializer configuration
- Is versioned semantically; each microservice pins a version, so schema evolution (new minor version of the library) is decoupled from each service's own release cadence
- Is published to the GitLab Package Registry (Maven repository type) via CI

```java
// event-schemas-lib: KafkaSerdeConfig.java
package com.yourorg.events.serde;

import io.apicurio.registry.serde.SerdeConfig;
import io.apicurio.registry.serde.strategy.TopicIdStrategy;

import java.util.Map;

public class KafkaSerdeConfig {

    public static Map<String, Object> producerConfig(String registryUrl) {
        return Map.of(
            SerdeConfig.REGISTRY_URL, registryUrl,
            SerdeConfig.ARTIFACT_RESOLVER_STRATEGY, TopicIdStrategy.class.getName(),
            SerdeConfig.AUTO_REGISTER_ARTIFACT, "false", // registration only via CI
            SerdeConfig.VALIDATION_ENABLED, "true"
        );
    }

    public static Map<String, Object> consumerConfig(String registryUrl) {
```

```
        return Map.of(
            SerdeConfig.REGISTRY_URL, registryUrl,
            SerdeConfig.CHECK_PERIOD_MS, "30000"
        );
    }
}
```

```
// OutboxEventPublisher.java - shared across services
package com.yourorg.events.outbox;

import io.apicurio.registry.serde.avro.AvroKafkaSerializer;
import org.apache.kafka.clients.producer.KafkaProducer;
import org.apache.kafka.clients.producer.ProducerRecord;
import org.apache.avro.specific.SpecificRecord;
import java.util.Map;

public class OutboxEventPublisher {

    private final KafkaProducer<String, SpecificRecord> producer;

    public OutboxEventPublisher(Map<String, Object> config) {
        this.producer = new KafkaProducer<>(config,
            new org.apache.kafka.common.serialization.StringSerializer(),
            new AvroKafkaSerializer<>());
    }

    public void publish(String topic, String key, SpecificRecord event) {
        producer.send(new ProducerRecord<>(topic, key, event));
    }
}
```

7. Transaction Outbox Pattern

The outbox table stores the serialized Avro payload alongside the transactional business write. A relay process reads the outbox and publishes to Kafka using the same serializer as the common library, guaranteeing identical schema resolution logic between the write path and the publish path. Because the Apicurio global schema ID is embedded in the serialized bytes at write time, later schema evolution never breaks already-queued outbox rows.

```
CREATE TABLE outbox_event (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    aggregate_type VARCHAR(100) NOT NULL,
    aggregate_id VARCHAR(100) NOT NULL,
    event_type VARCHAR(100) NOT NULL,
    topic VARCHAR(100) NOT NULL,
    schema_artifact_id VARCHAR(100) NOT NULL,
    schema_global_id BIGINT,
    payload BYTEA NOT NULL,
    created_at TIMESTAMPTZ DEFAULT now(),
    published BOOLEAN DEFAULT false
);
```

```java
@ApplicationScoped
public class OrderService {

    @Inject EntityManager em;
    @Inject AvroKafkaSerializer<OrderCreated> serializer;

    @Transactional
    public void createOrder(Order order) {
        em.persist(order); // business write

        OrderCreated event = OrderCreated.newBuilder()
            .setOrderId(order.getId())
            .setStatus("CREATED")
            .build();

        byte[] payload = serializer.serialize("order-events", event);

        OutboxEvent outbox = new OutboxEvent();
        outbox.setAggregateType("Order");
        outbox.setAggregateId(order.getId().toString());
        outbox.setEventType("OrderCreated");
        outbox.setTopic("order-events");
        outbox.setPayload(payload);
        em.persist(outbox); // same transaction as business write
    }
}
```

```java
@ApplicationScoped
public class OutboxRelay {

    @Inject EntityManager em;
    @Inject @Channel("order-events-out") Emitter<byte[]> emitter;

    @Scheduled(every = "5s")
    @Transactional
    public void relay() {
        List<OutboxEvent> pending = em.createQuery(
            "SELECT o FROM OutboxEvent o WHERE o.published = false ORDER BY o.createdAt",
            OutboxEvent.class).setMaxResults(100).getResultList();

        for (OutboxEvent evt : pending) {
            emitter.send(evt.getPayload());
            evt.setPublished(true);
        }
    }
}
```

8. GitLab CI/CD — Schema Repository Pipeline (No Maven Plugin, REST API Driven)

```yaml
stages:
  - lint
  - compat-check
  - register
  - promote

variables:
  APICURIO_DEV: "http://apicurio-dev.internal:8080/apis/registry/v3"
  APICURIO_STG: "http://apicurio-stg.internal:8080/apis/registry/v3"
  APICURIO_PRD: "http://apicurio-prd.internal:8080/apis/registry/v3"
  GROUP_ID: "order-events"

# ---------- LINT ----------
lint-avro:
  stage: lint
  image: registry.gitlab.com/yourorg/tools/avro-tools:latest
  rules:
    - changes: ["avro/**/*.avsc"]
  script:
    - for f in $(find avro -name "*.avsc"); do
        echo "Validating $f";
        avro-tools compile schema "$f" /tmp/out || exit 1;
      done

lint-protobuf:
  stage: lint
  image: bufbuild/buf:latest
  rules:
    - changes: ["protobuf/**/*.proto"]
  script:
    - buf lint protobuf/

lint-openapi:
  stage: lint
  image: stoplight/spectral:latest
  rules:
    - changes: ["openapi/**/*.yaml"]
  script:
    - spectral lint openapi/**/*.yaml

# ---------- COMPATIBILITY CHECK (dry-run against registry, MR only) ----------
compat-check-avro:
  stage: compat-check
  image: curlimages/curl:latest
  rules:
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'
      changes: ["avro/**/*.avsc"]
  script:
    - |
      for f in $(git diff --name-only origin/main...HEAD -- 'avro/**/*.avsc'); do
```

```
        ARTIFACT_ID=$(basename "$f" .avsc)
        echo "Checking compatibility for $ARTIFACT_ID"
        RESPONSE=$(curl -s -o /tmp/resp.json -w "%{http_code}" \
          -X PUT "${APICURIO_DEV}/groups/${GROUP_ID}/artifacts/${ARTIFACT_ID}/test" \
          -H "Content-Type: application/json" \
          --data-binary @"$f")
        if [ "$RESPONSE" != "204" ]; then
          echo "COMPATIBILITY FAILURE for $ARTIFACT_ID:"
          cat /tmp/resp.json
          exit 1
        fi
      done
  # Automated pre-merge gate; MR approval (checker) is enforced via
  # GitLab protected-branch settings, not this job.

# ---------- REGISTER (auto, on merge to main -> dev group) ----------
register-dev:
  stage: register
  image: curlimages/curl:latest
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
  script:
    - |
      for f in $(find avro -name "*.avsc"); do
        ARTIFACT_ID=$(basename "$f" .avsc)
        curl -s -f -X POST "${APICURIO_DEV}/groups/${GROUP_ID}/artifacts" \
          -H "Content-Type: application/json" \
          -H "X-Registry-ArtifactId: ${ARTIFACT_ID}" \
          -H "X-Registry-ArtifactType: AVRO" \
          -H "X-Registry-Version: ${CI_COMMIT_SHORT_SHA}" \
          --data-binary @"$f"
      done
    - |
      for f in $(find protobuf -name "*.proto"); do
        ARTIFACT_ID=$(basename "$f" .proto)
        curl -s -f -X POST "${APICURIO_DEV}/groups/${GROUP_ID}/artifacts" \
          -H "Content-Type: application/x-protobuf" \
          -H "X-Registry-ArtifactId: ${ARTIFACT_ID}" \
          -H "X-Registry-ArtifactType: PROTOBUF" \
          -H "X-Registry-Version: ${CI_COMMIT_SHORT_SHA}" \
          --data-binary @"$f"
      done
    - |
      for f in $(find openapi -name "*.yaml"); do
        ARTIFACT_ID=$(basename "$f" .yaml)
        curl -s -f -X POST "${APICURIO_DEV}/groups/${GROUP_ID}/artifacts" \
          -H "Content-Type: application/yaml" \
          -H "X-Registry-ArtifactId: ${ARTIFACT_ID}" \
          -H "X-Registry-ArtifactType: OPENAPI" \
          -H "X-Registry-Version: ${CI_COMMIT_SHORT_SHA}" \
          --data-binary @"$f"
      done
  environment:
```

```
    name: dev

# ---------- PROMOTE (manual, checker gate at registry level) ----------
promote-staging:
  stage: promote
  image: curlimages/curl:latest
  needs: ["register-dev"]
  when: manual
  environment:
    name: staging
  script:
    - |
      for f in $(find avro -name "*.avsc"); do
        ARTIFACT_ID=$(basename "$f" .avsc)
        curl -s -f -X POST "${APICURIO_STG}/groups/${GROUP_ID}/artifacts" \
          -H "Content-Type: application/json" \
          -H "X-Registry-ArtifactId: ${ARTIFACT_ID}" \
          -H "X-Registry-Version: ${CI_COMMIT_SHORT_SHA}" \
          --data-binary @"$f"
      done

promote-prod:
  stage: promote
  image: curlimages/curl:latest
  needs: ["promote-staging"]
  when: manual
  environment:
    name: production
  script:
    - |
      for f in $(find avro -name "*.avsc"); do
        ARTIFACT_ID=$(basename "$f" .avsc)
        curl -s -f -X POST "${APICURIO_PRD}/groups/${GROUP_ID}/artifacts" \
          -H "Content-Type: application/json" \
          -H "X-Registry-ArtifactId: ${ARTIFACT_ID}" \
          -H "X-Registry-Version: ${CI_COMMIT_SHORT_SHA}" \
          --data-binary @"$f"
      done
```

GitLab settings enforcing maker-checker here:

- Protected branch `main`: 2 approvals required, "prevent approval by author," CODEOWNERS = `schema-owners` team.
- `promote-staging` / `promote-prod` are `when: manual` and restricted via Protected Environments so only a designated Release Approver group can trigger deployment — a second, independent checker at registry-promotion time.

9. GitLab CI/CD — Per-Microservice Pipeline

```
stages: [build, test, image, deploy-dev, deploy-staging, deploy-prod]

build:
  stage: build
```

```
  image: maven:3.9-eclipse-temurin-17
  script:
    - mvn -B package -DskipTests

test:
  stage: test
  image: maven:3.9-eclipse-temurin-17
  script:
    - mvn -B test

image:
  stage: image
  image: docker:24
  services: [docker:24-dind]
  script:
    - docker build -t $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA -f src/main/docker/Dockerfile.jvm
    - docker push $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA

deploy-dev:
  stage: deploy-dev
  image: alpine/helm:3.14.0
  environment: { name: dev }
  script:
    - helm upgrade --install order-service ./helm/order-service
        -f ./helm/order-service/values-dev.yaml
        --set image.tag=$CI_COMMIT_SHORT_SHA
        --namespace dev
        --wait --timeout 3m

deploy-staging:
  stage: deploy-staging
  image: alpine/helm:3.14.0
  when: manual
  needs: ["deploy-dev"]
  environment: { name: staging }
  script:
    - helm upgrade --install order-service ./helm/order-service
        -f ./helm/order-service/values-staging.yaml
        --set image.tag=$CI_COMMIT_SHORT_SHA
        --namespace staging
        --wait --timeout 3m

deploy-prod:
  stage: deploy-prod
  image: alpine/helm:3.14.0
  when: manual
  needs: ["deploy-staging"]
  environment: { name: production }
  script:
    - helm upgrade --install order-service ./helm/order-service
        -f ./helm/order-service/values-prod.yaml
        --set image.tag=$CI_COMMIT_SHORT_SHA
        --namespace prod
```

```
        --wait --timeout 5m
    - helm history order-service -n prod --max 3
```

`deploy-staging` and `deploy-prod` sit under Protected Environments with a required-approvers list, giving maker-checker on the deployment side too — separate from schema-side approval. Rollback is `helm rollback order-service <revision> -n prod`.

10. Kubernetes Delivery — Helm Chart

```
order-service/
├── Chart.yaml
├── values.yaml
├── values-dev.yaml
├── values-staging.yaml
├── values-prod.yaml
└── templates/
    ├── deployment.yaml
    ├── service.yaml
    ├── configmap.yaml
    ├── secret.yaml
    ├── hpa.yaml
    └── _helpers.tpl
```

```yaml
# Chart.yaml
apiVersion: v2
name: order-service
description: Order service Quarkus microservice
version: 0.1.0
appVersion: "1.0.0"
```

```yaml
# values.yaml (defaults)
replicaCount: 2

image:
  repository: registry.gitlab.com/yourorg/order-service
  tag: latest
  pullPolicy: IfNotPresent

resources:
  requests:
    cpu: 250m
    memory: 384Mi
  limits:
    cpu: 1
    memory: 512Mi

apicurio:
  registryUrl: "http://apicurio-dev.internal:8080/apis/registry/v3"
  group: order-events

kafka:
  bootstrapServers: "kafka-dev.internal:9092"
  topic: order-events

quarkus:
  profile: dev

livenessProbe:
  path: /q/health/live
  initialDelaySeconds: 15
readinessProbe:
```

```
  path: /q/health/ready
  initialDelaySeconds: 10

autoscaling:
  enabled: false
  minReplicas: 2
  maxReplicas: 6
  targetCPUUtilizationPercentage: 75
```

```
# values-prod.yaml (override)
replicaCount: 4

image:
  tag: "" # injected by CI via --set

apicurio:
  registryUrl: "http://apicurio-prd.internal:8080/apis/registry/v3"

kafka:
  bootstrapServers: "kafka-prod.internal:9092"

quarkus:
  profile: prod

autoscaling:
  enabled: true
```

```
# templates/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Chart.Name }}
  namespace: {{ .Release.Namespace }}
  labels:
    app: {{ .Chart.Name }}
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app: {{ .Chart.Name }}
  template:
    metadata:
      labels:
        app: {{ .Chart.Name }}
    spec:
      containers:
        - name: {{ .Chart.Name }}
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          imagePullPolicy: {{ .Values.image.pullPolicy }}
          ports:
            - containerPort: 8080
          envFrom:
            - configMapRef:
                name: {{ .Chart.Name }}-config
```

```
            - secretRef:
                name: {{ .Chart.Name }}-secret
          resources:
            {{- toYaml .Values.resources | nindent 12 }}
          livenessProbe:
            httpGet:
              path: {{ .Values.livenessProbe.path }}
              port: 8080
            initialDelaySeconds: {{ .Values.livenessProbe.initialDelaySeconds }}
          readinessProbe:
            httpGet:
              path: {{ .Values.readinessProbe.path }}
              port: 8080
            initialDelaySeconds: {{ .Values.readinessProbe.initialDelaySeconds }}
```

```
# templates/configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: {{ .Chart.Name }}-config
  namespace: {{ .Release.Namespace }}
data:
  QUARKUS_PROFILE: {{ .Values.quarkus.profile | quote }}
  MP_MESSAGING_CONNECTOR_APICURIO_AVRO_REGISTRY_URL: {{ .Values.apicurio.registryUrl | quote }}
  KAFKA_BOOTSTRAP_SERVERS: {{ .Values.kafka.bootstrapServers | quote }}
  APP_KAFKA_TOPIC: {{ .Values.kafka.topic | quote }}
```

```
# templates/hpa.yaml
{{- if .Values.autoscaling.enabled }}
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: {{ .Chart.Name }}
  namespace: {{ .Release.Namespace }}
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: {{ .Chart.Name }}
  minReplicas: {{ .Values.autoscaling.minReplicas }}
  maxReplicas: {{ .Values.autoscaling.maxReplicas }}
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: {{ .Values.autoscaling.targetCPUUtilizationPercentage }}
{{- end }}
```

10.1 Kustomize Alternative

`k8s/`

```
├── base/
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── configmap.yaml
│   └── kustomization.yaml
└── overlays/
    ├── dev/kustomization.yaml
    ├── staging/kustomization.yaml
    └── prod/kustomization.yaml
```

```yaml
# base/kustomization.yaml
resources:
  - deployment.yaml
  - service.yaml
  - configmap.yaml
```

```yaml
# overlays/prod/kustomization.yaml
resources:
  - ../../base
namespace: prod
replicas:
  - name: order-service
    count: 4
images:
  - name: order-service
    newName: registry.gitlab.com/yourorg/order-service
    newTag: PLACEHOLDER
patches:
  - path: patch-resources.yaml
```

```yaml
# CI job (Kustomize path)
deploy-prod:
  stage: deploy-prod
  image: registry.gitlab.com/yourorg/tools/kustomize-kubectl:latest
  when: manual
  environment: { name: production }
  script:
    - cd k8s/overlays/prod
    - kustomize edit set image order-service=$CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA
    - kustomize build . | kubectl apply -f -
    - kubectl rollout status deployment/order-service -n prod
```

Helm is preferable when per-environment config differences (registry URL, Kafka brokers, replica counts) are numerous, since Helm's values files map naturally onto that; Kustomize suits teams that prefer plain-YAML overlays without a templating language.

11. Apicurio Compatibility Rules — Declarative Configuration

```
POST /apis/registry/v3/admin/rules
Content-Type: application/json

{
```

```
  "ruleType": "COMPATIBILITY",
  "config": "BACKWARD"
}
```

```
# Per-group rule override
curl -X POST "${APICURIO_URL}/groups/order-events/rules" \
  -H "Content-Type: application/json" \
  -d '{
    "ruleType": "COMPATIBILITY",
    "config": "BACKWARD_TRANSITIVE"
  }'
```

```
# Per-artifact override
curl -X POST "${APICURIO_URL}/groups/order-events/artifacts/Payment/rules" \
  -H "Content-Type: application/json" \
  -d '{
    "ruleType": "COMPATIBILITY",
    "config": "FULL"
  }'
```

```
# Validity rule (catches malformed schemas before compatibility runs)
curl -X POST "${APICURIO_URL}/groups/order-events/rules" \
  -H "Content-Type: application/json" \
  -d '{
    "ruleType": "VALIDITY",
    "config": "FULL"
  }'
```

12. Maker-Checker Trace Summary

Stage	Maker	Checker	Mechanism
Schema change	Dev opens MR	Schema owner approves	GitLab CODEOWNERS + protected branch
Compatibility	Automated	N/A (hard gate)	CI job hitting Apicurio `/test` endpoint
Registry promotion	Pipeline reaches manual stage	Release approver	GitLab Protected Environments
Service deployment	Pipeline reaches manual stage	Ops/release approver	GitLab Protected Environments
Outbox event	App writes transactionally	N/A (system-enforced)	DB transaction + relay

13. Fleet-Scale Rollout: 700 Microservices Without Source Code Changes

13.1 The Key Insight

In Quarkus (SmallRye Reactive Messaging), the Kafka serializer/deserializer class and the registry URL are externalized MicroProfile Config properties — not hardcoded Java. If services already use standard `@Incoming` / `@Outgoing` Kafka channels, the serializer can be swapped entirely through configuration:

```
mp.messaging.outgoing.order-events-out.serializer=io.apicurio.registry.serde.avro.AvroKafkaSeri
mp.messaging.outgoing.order-events-out.apicurio.registry.url=${APICURIO_REGISTRY_URL}
```

The real challenge becomes getting the Apicurio serde JAR onto the classpath and the right properties injected — for 700 services — without opening 700 source MRs.

13.2 Classpath Injection Without Touching Source

Option A — Shared base image layer (recommended at this scale). If every service's Dockerfile extends a common base image, bump that base image tag centrally to add the Apicurio serde jars. All 700 services inherit the new classpath on their next rebuild — no per-repo MR for the jar.

```
FROM registry.access.redhat.com/ubi8/openjdk-17:latest AS apicurio-layer
COPY apicurio-avro-serdes-all.jar /deployments/lib/apicurio-serde.jar
COPY common-event-schemas.jar /deployments/lib/common-schemas.jar
```

Option B — Init container (for heterogeneous Dockerfiles). Drop jars onto a shared volume and extend the classpath via `JAVA_OPTS_APPEND`, purely as a Helm chart / K8s manifest change:

```
initContainers:
  - name: apicurio-jar-injector
    image: registry.gitlab.com/yourorg/apicurio-serde-payload:latest
    command: ["sh", "-c", "cp /payload/*.jar /shared-libs/"]
    volumeMounts:
      - name: shared-libs
        mountPath: /shared-libs
containers:
  - name: order-service
    env:
      - name: JAVA_OPTS_APPEND
        value: "-cp /shared-libs/*"
    volumeMounts:
      - name: shared-libs
        mountPath: /shared-libs
volumes:
  - name: shared-libs
    emptyDir: {}
```

13.3 Centralized Config Delivery

Quarkus config resolution gives environment variables priority over `application.properties`, so config can be delivered centrally via a shared Helm library chart rather than editing 700 properties files:

```
{{- define "apicurio.env" -}}
- name: MP_MESSAGING_OUTGOING_ORDER_EVENTS_OUT_SERIALIZER
  value: "io.apicurio.registry.serde.avro.AvroKafkaSerializer"
- name: MP_MESSAGING_OUTGOING_ORDER_EVENTS_OUT_APICURIO_REGISTRY_URL
  value: {{ .Values.apicurio.registryUrl | quote }}
- name: MP_MESSAGING_INCOMING_ORDER_EVENTS_IN_DESERIALIZER
  value: "io.apicurio.registry.serde.avro.AvroKafkaDeserializer"
{{- end }}
```

One Helm library-chart version bump propagates to all 700 services on their next deploy — again, zero source diffs.

13.4 Maker-Checker Reframed at Fleet Scale

Gate	What's being approved	Where	Checker
G1 – Base image / shared jar	New Apicurio serde version in the common base image	`platform-base-image` repo MR	Platform team CODEOWNERS
G2 – Helm library chart	Env-var wiring / serializer class change	`helm-common-library` repo MR	Platform + Kafka team CODEOWNERS
G3 – Schema artifact	`.avsc` / `.proto` / `.yaml` change	`schema-registry` repo MR	Domain/schema owners
G4 – Rollout wave	Which services adopt the change, in what order	`fleet-rollout-config` repo	Release manager, manual protected environment

This collapses the approval surface from 700 individual reviews to four centralized gates.

13.5 Fleet-Wide Rollout via GitOps

```
# fleet-rollout-config/services.yaml
services:
  - name: order-service
    chartVersion: "2.3.0"
    wave: 1
  - name: payment-service
    chartVersion: "2.1.0"   # not yet migrated
    wave: 2
  - name: inventory-service
    chartVersion: "2.3.0"
    wave: 1
```

```
rollout-wave-1:
  stage: rollout
  when: manual   # checker gate — release manager approves the wave
  environment: { name: production }
  script:
    - |
      for svc in $(yq '.services[] | select(.wave==1) | .name' fleet-rollout-config/services.ya
```

```
    VERSION=$(yq ".services[] | select(.name==\"$svc\") | .chartVersion" fleet-rollout-conf
    helm upgrade --install $svc oci://registry.gitlab.com/yourorg/charts/$svc \
      --version $VERSION -n prod --wait --timeout 3m
  done
```

A single MR to `fleet-rollout-config`, bumping `chartVersion` for the services in a wave, becomes the maker-checker unit for the whole migration — a canary wave of 5–10 services first, validated against Kafka lag / error rates / Apicurio hit metrics, then widened wave by wave.

13.6 Feature-Flag Safety Net

```
- name: SCHEMA_SERDE_MODE
  value: {{ .Values.apicurio.enabled | ternary "apicurio" "legacy" }}
```

If `legacy`, the chart renders the old serializer configuration instead, so rollback of a bad wave is a Helm values flip plus rolling restart — not a source code revert.

14. Fleet Audit: Config-Only vs Code-Touch Bucketing

This retrofit only works cleanly for services already using standard SmallRye Reactive Messaging Kafka channels. Services that hand-roll their own `KafkaProducer` / `KafkaConsumer` construction in Java, or implement custom `Serializer<T>` classes, need actual source code changes. An audit across all 700 repos sizes these buckets before committing to the wave plan.

14.1 Enumerate All Repos (GitLab API)

```
#!/usr/bin/env bash
# discover-services.sh
set -euo pipefail

GITLAB_URL="https://gitlab.yourorg.com"
GROUP_ID="12345"
TOKEN="${GITLAB_TOKEN}"

PAGE=1
> repos.txt
while true; do
  RESP=$(curl -s --header "PRIVATE-TOKEN: ${TOKEN}" \
    "${GITLAB_URL}/api/v4/groups/${GROUP_ID}/projects?include_subgroups=true&per_page=100&page=
  COUNT=$(echo "$RESP" | jq 'length')
  [ "$COUNT" -eq 0 ] && break
  echo "$RESP" | jq -r '.[] | "\(.id)|\(.path_with_namespace)|\(.http_url_to_repo)"' >> repos.t
  PAGE=$((PAGE + 1))
done

echo "Discovered $(wc -l < repos.txt) repos"
```

14.2 Classify Each Repo's Kafka Integration Pattern

```
#!/usr/bin/env bash
# audit-kafka-pattern.sh
set -euo pipefail

TOKEN="${GITLAB_TOKEN}"
WORKDIR="/tmp/audit-workspace"
RESULTS="audit-results.csv"
mkdir -p "$WORKDIR"

echo "repo,category,reactive_messaging,raw_kafka_producer,raw_kafka_consumer,custom_serializer,

while IFS='|' read -r id path url; do
  REPO_DIR="${WORKDIR}/${id}"
  echo "Auditing: $path"

  rm -rf "$REPO_DIR"
  git clone --depth 1 --quiet \
    "https://oauth2:${TOKEN}@${url#https://}" "$REPO_DIR" 2>/dev/null || {
      echo "$path,ERROR,,,,,,,clone_failed" >> "$RESULTS"
```

```
      continue
  }

  cd "$REPO_DIR"

  # Signal 1: Reactive Messaging config-based Kafka usage
  REACTIVE=$(grep -rl "mp\.messaging\.\(incoming\|outgoing\)" \
    --include="*.properties" --include="*.yaml" --include="*.yml" . 2>/dev/null | wc -l)

  # Signal 2: Raw KafkaProducer/Consumer instantiation (code-touch needed)
  RAW_PRODUCER=$(grep -rl "new KafkaProducer\|KafkaProducer<" \
    --include="*.java" . 2>/dev/null | wc -l)
  RAW_CONSUMER=$(grep -rl "new KafkaConsumer\|KafkaConsumer<" \
    --include="*.java" . 2>/dev/null | wc -l)

  # Signal 3: Existing custom serializer classes (code-touch, needs replacement)
  CUSTOM_SERIALIZER=$(grep -rl "implements Serializer<\|implements Deserializer<" \
    --include="*.java" . 2>/dev/null | wc -l)

  # Signal 4: Already using Confluent/other Avro serde (needs swap, not raw code touch)
  AVRO_LIB=$(grep -rl "io\.confluent\.kafka\.serializers\|KafkaAvroSerializer\|KafkaAvroDeseria
    --include="*.properties" --include="*.yaml" --include="*.java" --include="pom.xml" . 2>/dev

  if [ "$RAW_PRODUCER" -gt 0 ] || [ "$RAW_CONSUMER" -gt 0 ] || [ "$CUSTOM_SERIALIZER" -gt 0 ];
    CATEGORY="NEEDS_CODE_TOUCH"
    CONFIDENCE="high"
    NOTES="hand-rolled Kafka client or custom serializer detected"
  elif [ "$REACTIVE" -gt 0 ] && [ "$AVRO_LIB" -gt 0 ]; then
    CATEGORY="CONFIG_ONLY_SERDE_SWAP"
    CONFIDENCE="high"
    NOTES="uses reactive messaging + existing Avro serde, just swap serializer class"
  elif [ "$REACTIVE" -gt 0 ]; then
    CATEGORY="CONFIG_ONLY_NEW_SERDE"
    CONFIDENCE="medium"
    NOTES="reactive messaging present, no existing Avro serde - verify payload format manually"
  else
    CATEGORY="UNKNOWN_NO_KAFKA_DETECTED"
    CONFIDENCE="low"
    NOTES="no Kafka usage pattern found - verify manually"
  fi

  echo "$path,$CATEGORY,$REACTIVE,$RAW_PRODUCER,$RAW_CONSUMER,$CUSTOM_SERIALIZER,$AVRO_LIB,$CON

  cd - > /dev/null
  rm -rf "$REPO_DIR"

done < repos.txt

echo "Audit complete. Results in $RESULTS"
```

14.3 Refinement Pass — Reduce False Positives

```
#!/usr/bin/env bash
# refine-audit.sh
set -euo pipefail

# Re-check "NEEDS_CODE_TOUCH" entries: exclude if raw KafkaProducer only
# appears in test sources rather than src/main
while IFS=',' read -r path category rest; do
  [ "$category" != "NEEDS_CODE_TOUCH" ] && continue
  REPO_DIR="/tmp/audit-workspace/recheck-$(echo "$path" | md5sum | cut -d' ' -f1)"
  MAIN_HITS=$(grep -rl "new KafkaProducer\|new KafkaConsumer" \
    --include="*.java" "$REPO_DIR/src/main" 2>/dev/null | wc -l)
  if [ "$MAIN_HITS" -eq 0 ]; then
    echo "RECLASSIFY: $path - raw Kafka usage only in tests, likely config-only migratable"
  fi
done < audit-results.csv
```

14.4 Summary Report

```
#!/usr/bin/env bash
# summarize.sh
echo "=== Fleet Migration Bucket Summary ==="
echo
echo "Total repos audited: $(($(wc -l < audit-results.csv) - 1))"
echo
echo "By category:"
tail -n +2 audit-results.csv | cut -d',' -f2 | sort | uniq -c | sort -rn
echo
echo "High-confidence config-only candidates (safe for Wave 1):"
awk -F',' '$2=="CONFIG_ONLY_SERDE_SWAP" && $8=="high"' audit-results.csv | wc -l
echo
echo "Needs manual engineering review:"
awk -F',' '$2=="NEEDS_CODE_TOUCH" || $2=="UNKNOWN_NO_KAFKA_DETECTED"' audit-results.csv | wc -l
```

14.5 Expected Output Shape

```
repo,category,reactive_messaging,raw_kafka_producer,raw_kafka_consumer,custom_serializer,avro_l
team-a/order-service,CONFIG_ONLY_SERDE_SWAP,1,0,0,0,1,high,"uses reactive messaging + existing
team-b/legacy-payment-svc,NEEDS_CODE_TOUCH,0,2,1,1,0,high,"hand-rolled Kafka client or custom s
team-c/notification-service,CONFIG_ONLY_NEW_SERDE,1,0,0,0,0,medium,"reactive messaging present,
team-d/reporting-service,UNKNOWN_NO_KAFKA_DETECTED,0,0,0,0,0,low,"no Kafka usage pattern found
```

14.6 How the Audit Feeds the Wave Plan

- **`CONFIG_ONLY_SERDE_SWAP`** (high confidence) → straight into Wave 1, purely a Helm/env-var migration, no repo MR at all.
- **`CONFIG_ONLY_NEW_SERDE`** (medium) → Wave 2, same mechanism but a human should verify the actual payload schema before assigning an `.avsc`.
- **`NEEDS_CODE_TOUCH`** → a separate backlog; these are the only services where a developer touches Java code, replacing hand-rolled producer/consumer construction with reactive messaging channels, or removing a custom `Serializer<T>` class.

- **`UNKNOWN`** → a triage list — likely non-Kafka services, or services using a different messaging pattern (JMS, AMQP) worth investigating separately; excluded from migration scope until confirmed.

In most large estates this splits roughly 70/20/10 (config-only / needs-review / needs-code), which indicates up front whether the zero-code-change plan covers the bulk of the fleet or whether a larger engineering lift should be expected.

A further refinement worth adding is a `pom.xml` dependency check — confirming the `quarkus-messaging-kafka` extension is actually present versus a service that manages Kafka client dependencies manually outside the Quarkus extension model — as an additional migratability signal.